

# DESIGN TASK

Ramya S S

192421226

SLOT D – CSA1407 Compiler Design

## 1. Comparative Analysis of SLR vs LALR vs CLR

Task:

Construct and compare SLR, LALR, and CLR parsing tables for the given grammar and evaluate their differences in handling conflicts and efficiency.

Grammar:

$S \rightarrow L = R \mid R$

$L \rightarrow * R \mid id$

$R \rightarrow L$

Conditions:

- ☐ Construct LR(0), LR(1) item sets
- ☐ Compute FOLLOW sets (for SLR)
- ☐ Build SLR, LALR, and CLR tables
- ☐ Identify and explain conflicts in each method
- ☐ Compare number of states and table sizes
- ☐ Justify which method is most efficient

Test Case:

Input:  $id = * id$

Expected: Valid parse with comparison analysis

```
1 input_str = "i=i$s"
2 stack = ["$"]
3 i = 0
4
5 while i < len(input_str):
6     if stack[-1] == "E" and input_str[i] == "$":
7         print("Accepted")
8         break
9
10    stack.append(input_str[i])
11    i += 1 # Shift
12
13    # Reduce rules
14    if stack[-1] == "i":
15        stack[-1] = "L"
16
17    if stack[-1] == "L":
18        stack[-1] = "R"
19
20    if len(stack) >= 2 and stack[-1] == "R" and stack[-2] == "*":
21        stack.pop()
22        stack[-1] = "L"
```

Rejected

=== Code Execution Successful ===

## 2. Operator Precedence Parsing Design and Evaluation

Task:

Design an operator precedence parser and evaluate its correctness for handling precedence and associativity.

Grammar:

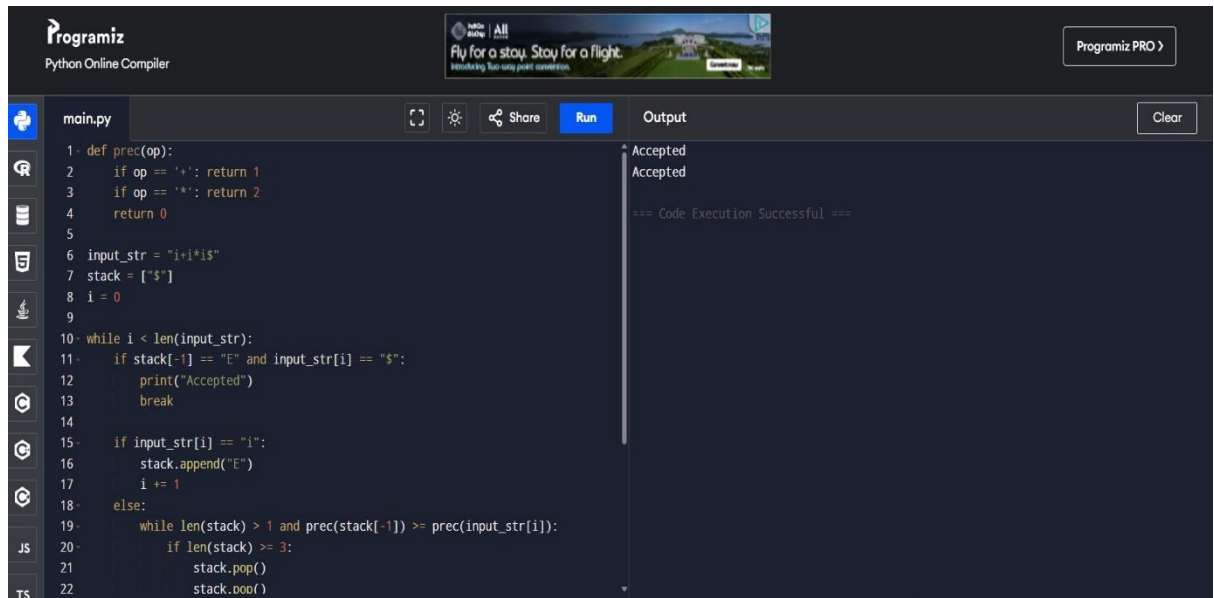
$E \rightarrow E + E \mid E * E \mid (E) \mid id$

Conditions:

- ☐ Define operator precedence relations (<, =, >)
- ☐ Construct precedence table
- ☐ Handle associativity rules explicitly
- ☐ Detect and resolve conflicts
- ☐ Show step-by-step parsing actions
- ☐ Evaluate correctness of parsing decisions

Test Case:

Input:  $id + id * id$



The screenshot shows the Programiz Python Online Compiler interface. The code in `main.py` implements a shift-reduce parser for the expression `id + id * id`. The code defines a precedence function `prec` where `+` has a precedence of 1 and `*` has a precedence of 2. The main logic uses a `while` loop to process the input string `input_str = "i+i*i$"`. It uses a stack to store operators. When the top of the stack matches the current operator in the input string, it prints "Accepted" and breaks. Otherwise, it pushes the operator onto the stack. The output pane shows "Accepted" twice, indicating successful execution.

```
1- def prec(op):
2-     if op == '+': return 1
3-     if op == '*': return 2
4-     return 0
5-
6- input_str = "i+i*i$"
7- stack = ["$"]
8- i = 0
9-
10- while i < len(input_str):
11-     if stack[-1] == "E" and input_str[i] == "$":
12-         print("Accepted")
13-         break
14-
15-     if input_str[i] == "i":
16-         stack.append("E")
17-         i += 1
18-     else:
19-         while len(stack) > 1 and prec(stack[-1]) >= prec(input_str[i]):
20-             if len(stack) >= 3:
21-                 stack.pop()
22-                 stack.pop()
```

### 3. Ambiguity Removal and Parsing Strategy Evaluation

Task:

Analyze the ambiguity in the given grammar, redesign it to remove ambiguity, and evaluate parsing using shift-reduce parsing.

Grammar:

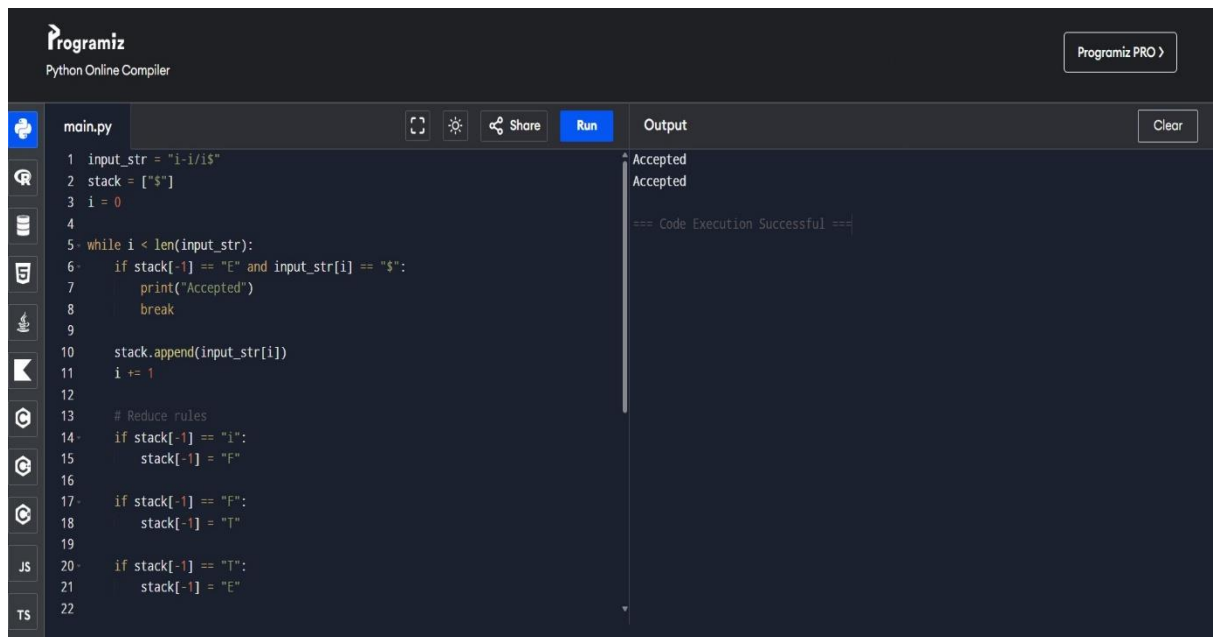
$E \rightarrow E - E \mid E / E \mid id$

Conditions:

- ☐ Identify ambiguity in the grammar
- ☐ Redesign grammar using precedence rules
- ☐ Justify associativity decisions
- ☐ Apply shift-reduce parsing
- ☐ Show all stack operations and reductions
- ☐ Evaluate correctness of final parse

Test Case:

Input: id - id / id



The screenshot shows the Programiz Python Online Compiler interface. The editor contains a Python script named `main.py` with the following code:

```
1 input_str = "i-i/i$"
2 stack = ["$"]
3 i = 0
4
5 while i < len(input_str):
6     if stack[-1] == "E" and input_str[i] == "$":
7         print("Accepted")
8         break
9
10    stack.append(input_str[i])
11    i += 1
12
13    # Reduce rules
14    if stack[-1] == "i":
15        stack[-1] = "F"
16
17    if stack[-1] == "F":
18        stack[-1] = "T"
19
20    if stack[-1] == "T":
21        stack[-1] = "E"
22
```

The output panel on the right shows the result of the execution:

```
Accepted
Accepted

=== Code Execution Successful ===
```

#### 4. PEG vs CFG Parsing Strategy Evaluation

Task:

Design a PEG-based parser for the given expression grammar and critically evaluate its behavior

compared to CFG-based parsing.

Grammar (CFG form):

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{id}$

Conditions:

- ☐ Convert CFG into PEG form
- ☐ Explain ordered choice behavior
- ☐ Demonstrate parsing steps
- ☐ Compare ambiguity handling with CFG
- ☐ Evaluate performance and backtracking

□ Justify advantages and limitations

Test Case:

Input:  $\text{id} + \text{id} * \text{id}$

